Code Similarity Platform

Installation Manual

Version: 1.0

Prepared by: Group 16 — Muhammad Nabeel Waheed, Thomas Neal, Ghassan Balouze, Kev Akpinar, Abdel Zahran, Ty Mabee

Kool Group Name: Last Minute Warriors

Course: COSC 4P02 — Course Project

2026-04-24

For system administrators, teaching assistants, and deployment engineers

Contents

Chapter 1

Purpose of This Document

1.1 Scope

This manual explains how to assemble, configure, install, and start the Similarity App from a fresh repository checkout. It focuses on practical setup work:

- preparing the host environment,
- configuring environment variables,
- initializing the database,
- building and wiring the Rust engine into the worker,
- starting the services in development,
- starting the stack in a Docker-based production-style deployment,
- and verifying that the integrated system works end-to-end.

It deliberately avoids repeating detailed architecture internals, deep algorithm analysis, maintenance procedures, and troubleshooting runbooks already covered in the other project documents.

1.2 Target Audience

This manual is written for:

- system administrators preparing a host or server,
- TAs or course staff running a local development or staging deployment,
- and deployers assembling the full runtime stack for demonstration or production-style use.

Chapter 2

System Overview

2.1 Concise Runtime Topology

At runtime, the system is composed of six cooperating parts:

- **Frontend**: Next.js web application for students, professors, and TAs.
- **API**: NestJS + Fastify service that exposes HTTP endpoints, handles authentication, receives uploads, and publishes background jobs.
- **Worker**: BullMQ consumer that prepares uploaded archives and launches similarity analysis jobs.
- **Engine**: standalone Rust CLI binary invoked by the worker over stdin/stdout.
- **Database**: PostgreSQL storing users, assignments, uploads, submissions, templates, comparison runs, pair results, and sessions.
- **Queue and Shared Storage**: Redis transports jobs; a shared filesystem-backed object storage area holds uploaded and prepared artifacts.

2.2 How the Components Are Stitched Together

2.3 Practical Interaction Flow

The important installation fact is not the internal implementation of each service, but how they depend on one another during startup and runtime:

1. The **frontend** must know the API base URL.
2. The **API** must reach PostgreSQL and Redis.
3. The **worker** must reach PostgreSQL, Redis, the shared object-storage directory, and the Rust engine binary.
4. The **engine** is not a network service; it must exist as an executable file on the worker host or inside the worker container.

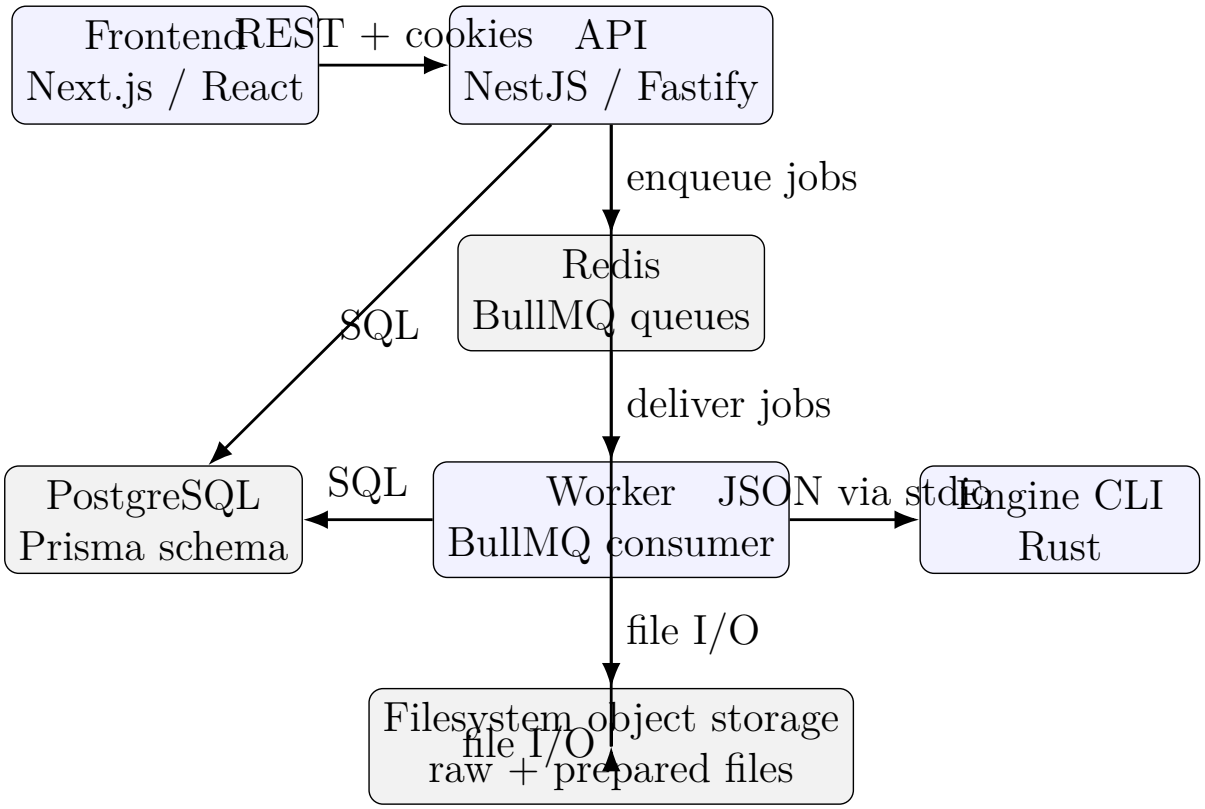

Figure 2.1: High-level installation and runtime assembly

5. The **API and worker** must share the same object-storage root so that uploaded archives written by one component can be read by the other.

Chapter 3

Environment and Prerequisites

3.1 Hardware Requirements

The repository documentation targets a single-node deployment suitable for course-scale use. Baseline recommendations are shown below.

Component	Minimum	Recommended
API server	1 vCPU / 512 MB RAM	2 vCPU / 1 GB RAM
Worker	1 vCPU / 512 MB RAM	2 vCPU / 1 GB RAM
PostgreSQL	1 vCPU / 512 MB RAM	2 vCPU / 2 GB RAM
Redis	128 MB RAM	512 MB RAM
Frontend	256 MB RAM	512 MB RAM
Object storage disk	5 GB	20 GB or more

The worker and engine are the most CPU-sensitive part of the stack. Comparison workload grows with the number of submission pairs, so installations intended for larger classes should budget CPU accordingly.

3.2 Supported Operating Systems

- **Docker deployment**: any recent Linux host with Docker Engine and Docker Compose.
- **Bare-metal/local development**: Debian 12, Ubuntu 22.04 LTS or newer are explicitly aligned with the container base images.
- **Developer workstations**: macOS and Windows can be used for repository work, but note the engine-binary path requirement described in Chapter 7.

3.3 Required Software

Software	Version family seen in repo	Use
Node.js	22.x	API, worker, frontend
npm	10.x class tooling	workspace dependency management
Rust toolchain	1.86+	engine build
PostgreSQL	16	relational database
Redis	7	BullMQ queue backend
Docker Engine	24+	container runtime
Docker Compose	2.20+	multi-service startup
Caddy	2.x	reverse proxy in production-style deployment

3.4 Network and Port Requirements

Service	Port	Protocol	Notes
Frontend (direct dev)	3000	TCP	Next.js application
API (direct dev)	3001	TCP	HTTP API and health endpoints
PostgreSQL	5432	TCP	API and worker database access
Redis	6379	TCP	API publishing, worker consumption
Caddy (prod-style)	80, 443	TCP	public reverse proxy entrypoint

PostgreSQL and Redis should not be publicly exposed in a production deployment.

Chapter 4

Repository Setup

4.1 Clone and Initial Layout

Clone the repository and enter the project root:

Listing 4.1: Clone and enter the repository

```
git clone <repository-url> Anti-Vibe-Coder
cd Anti-Vibe-Coder
```

4.2 Relevant Repository Structure

Only the following paths are essential for installation and startup:

Path	Setup role
`apps/web`	Next.js frontend workspace
`apps/api`	NestJS + Fastify API workspace
`apps/worker`	BullMQ worker workspace
`packages/shared`	shared TypeScript contracts, queue names, and storage utilities
`engine`	Rust workspace containing the engine CLI and supporting crates
`prisma`	Prisma schema, migrations, seed/bootstrap scripts
`docker-compose.yml`	multi-service Docker deployment definition
`Dockerfile.api`, `Dockerfile.worker`, `Dockerfile.web`	container build recipes
`Caddyfile`	reverse proxy routing for production-style deployment

4.3 Install Node Dependencies

From the repository root:

Listing 4.2: Install npm workspace dependencies

```
npm ci
```

This installs all workspace dependencies needed for the frontend, API, worker, and shared modules.

4.4 Generate Prisma Client

The API and worker both rely on the Prisma client. Generate it before building or starting services:

Listing 4.3: Generate the Prisma client

```
npm run prisma:generate
```

Chapter 5

Environment Configuration

5.1 Create the Environment File

The repository includes `.env.example`. Copy it to `.env` and fill in the required values:

Listing 5.1: Create the environment file

```
cp .env.example .env
```

5.2 Required Secrets for Public Uploads

Two security-sensitive values are mandatory for a real deployment:

1. a status-token secret used for public upload status links,
2. and an RSA key pair used for encrypted student identity submission.

Listing 5.2: Generate a public-upload status token secret

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Listing 5.3: Generate an RSA key pair for submission identity

```
node -e "
const crypto = require('crypto');
const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa', {
  modulusLength: 4096,
  publicKeyEncoding: { type: 'spki', format: 'pem' },
  privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
});
console.log('SUBMISSION_IDENTITY_KEY_ID=prod-key-001');
console.log('SUBMISSION_IDENTITY_PUBLIC_KEY_PEM_BASE64=' + Buffer.from(
    ↪ publicKey).toString('base64'));
console.log('SUBMISSION_IDENTITY_PRIVATE_KEY_PEM_BASE64=' + Buffer.from(
    ↪ privateKey).toString('base64'));
"
```

5.3 Environment Variable Reference

5.3.1 Shared and Database Variables

Variable	Required	Typical value	Effect
NODE_ENV	Optional	development or production	Controls runtime defaults and production checks.
DATABASE_URL	Yes	PostgreSQL connection string	Used by Prisma in the API and worker.
REDIS_URL	Prod: yes, Dev: defaulted	redis://localhost:6379 or container hostname	Queue transport for API and worker.

5.3.2 API and Frontend Variables

Variable	Required	Typical value	Effect
API_HOST	Optional	0.0.0.0	API bind address.
API_PORT	Optional	3001	API bind port.
CORS_ALLOWED_ORIGINS	Prod: yes, Dev: defaulted	http://localhost:3000 or public domain list	Allowed browser origins for frontend requests.
UPLOAD_MAX_FILE_SIZE_MB	Optional	250	Fastify multipart upload size limit.
NEXT_PUBLIC_API_BASE_URL	Prod: yes, Dev: recommended	http://localhost:3001 or public API URL	Compiled into the frontend API client.

5.3.3 Auth and Security Variables

Variable	Required	Typical value	Effect
AUTH_SESSION_COOKIE_NAME	Optional	similarity_session	Name of the session cookie.
AUTH_SESSION_TTL_HOURS	Optional	168	Session duration in hours.
AUTH_COOKIE_SECURE	Optional, but must be reviewed	false in dev, true in prod	Whether cookies require HTTPS.
AUTH_COOKIE_SAME_SITE	Optional	lax	SameSite policy for the auth cookie.
AUTH_COOKIE_DOMAIN	Optional	blank in dev, public domain in prod	Cookie domain scoping.
PUBLIC_UPLOAD_STATUS_TOKEN_SECRET	Prod: yes, Dev: defaulted	random 32-byte hex secret	Protects public upload status lookups.
SUBMISSION_IDENTITY_KEY_ID	Prod: yes, Dev: defaulted	key identifier	Labels the active submission-identity keypair.
SUBMISSION_IDENTITY_PUBLIC_KEY_PEM_BASE64	Prod: yes, Dev: generated at runtime	base64-encoded PEM	Public key served to browsers for identity encryption.
SUBMISSION_IDENTITY_PRIVATE_KEY_PEM_BASE64	Prod: yes, Dev: generated at runtime	base64-encoded PEM	Private key used by the API to reveal encrypted identity.

5.3.4 Worker, Engine, and Storage Variables

Variable	Required	Typical value	Effect
ENGINE_BINARY_PATH	Prod: yes, Dev: strongly recommended	local or container path to engine-cli	Worker checks this path before startup and spawns the binary from it.
ENGINE_VERSION	Optional	0.1.0	Stored with comparison-run metadata.
ENGINE_GST_MIN_MATCH_LENGTH	Optional	8 in checked-in install examples/config; code default 6	Minimum GST tile length requested by the worker/API configuration. Checked-in installation examples and compose configuration still explicitly set 8, while the current code default is 6.

ENGINE_MINIMUM_COMMENT_LENGTH	Optional	12 (configured/example value)	Minimum comment length passed into engine requests. Effective comment matching currently requires at least 35 normalized characters.
WORKER_UPLOAD_CONCURRENCY	Optional	2	Upload-preparation worker concurrency.
WORKER_COMPARISON_CONCURRENCY	Optional	1	Comparison worker concurrency.
OBJECT_STORAGE_MODE	Optional	filesystem	Only filesystem mode is supported by the codebase.
OBJECT_STORAGE_ROOT	Prod: yes, Dev: defaulted	./var/object-storage or mounted volume	Shared artifact directory for raw and prepared files.
ARCHIVE_MAX_DEPTH	Optional	8	Nested ZIP depth limit during extraction.
ARCHIVE_MAX_SOURCE_FILES	Optional	2000	Maximum allowed extracted source-file count.
ARCHIVE_MAX_EXPANDED_BYTES	Optional	52428800	Maximum total extracted archive size in bytes.

5.3.5 Bootstrap Variables

Variable	Required	Typical value	Effect
ADMIN_BOOTSTRAP_EMAIL	Required only for bootstrap	administrator email	Used by the bootstrap script to create the initial professor account.
ADMIN_BOOTSTRAP_PASSWORD	Required only for bootstrap	strong password	Used by the bootstrap script to set the initial professor password.

5.4 Practical Notes on Required vs Optional Values

- In **development**, several values have code defaults, but DATABASE_URL must still be set and ENGINE_BINARY_PATH should be set explicitly.
- In **production**, treat the security, API-origin, engine-path, and object-storage settings as mandatory even if the code has development fallbacks.
- The frontend compiles in NEXT_PUBLIC_API_BASE_URL; if it is wrong at build time, the browser will call the wrong API.

Chapter 6

Database Setup

6.1 Start PostgreSQL

You may use a local PostgreSQL installation or a Docker container. The simplest repository-aligned path is to start PostgreSQL with Docker Compose:

Listing 6.1: Start only PostgreSQL and Redis first

```
docker compose up -d postgres redis
```

6.2 Generate the Prisma Client

Listing 6.2: Generate Prisma client before migrations

```
npm run prisma:generate
```

6.3 Apply Migrations

For a fresh local development database:

Listing 6.3: Apply development migrations

```
npm run db:migrate:dev
```

For a deployment using committed migrations:

Listing 6.4: Apply committed migrations

```
npm run db:migrate:deploy
```

6.4 Optional Development Reset

If you need to recreate a local development database from scratch:

Listing 6.5: Reset the development database

```
npm run db:reset:dev
```

6.5 Bootstrap the Initial Professor Account

After migrations are applied, create the initial professor account:

Listing 6.6: Bootstrap the initial professor login

```
ADMIN_BOOTSTRAP_EMAIL=admin@example.com
ADMIN_BOOTSTRAP_PASSWORD=ChangeMe123!
npm run db:bootstrap-admin
```

This step is required before testing the professor workflow unless you use the public professor-signup flow instead.

Chapter 7

Engine Setup

7.1 Build the Rust Engine

The engine lives in the Rust workspace under `engine/`. Build it from the repository root with either a debug or release profile.

Listing 7.1: Build the engine in debug mode

```
cargo build --manifest-path engine/Cargo.toml
```

Listing 7.2: Build the engine in release mode

```
cargo build --manifest-path engine/Cargo.toml --release
```

7.2 Expected Binary Locations

After building:

- debug build: `engine/target/debug/engine-cli`
- release build: `engine/target/release/engine-cli`

Inside the production worker container, the Dockerfile copies the release binary to:

Listing 7.3: Engine path inside the worker container

```
/app/bin/engine-cli
```

7.3 How the Worker Connects to the Engine

The worker does not link to the engine as a library. Instead:

1. on startup, the worker calls `access()` on `ENGINE_BINARY_PATH`,
2. for each comparison job, it spawns the engine as a child process,
3. writes a JSON analysis request to stdin,

4. and parses the JSON response from stdout.

7.4 Required Configuration

Set `ENGINE_BINARY_PATH` in `.env` to the actual executable:

Listing 7.4: Recommended local engine path setting

```
ENGINE_BINARY_PATH=./engine/target/release/engine-cli
```

7.5 Common Pitfalls

- **Wrong path**: the worker will fail before processing jobs if the binary path is invalid.
- **Platform mismatch**: the worker's development fallback path in code points to a Windows-style `engine-cli.exe`. On Linux or macOS, set `ENGINE_BINARY_PATH` explicitly.
- **Forgotten rebuild**: if engine code changes, rebuild the binary before expecting worker behavior to change.
- **Permissions**: the binary must be executable by the user running the worker.

Chapter 8

Running the System in Development

8.1 Recommended Startup Sequence

The order below minimizes configuration and dependency errors.

1. Start PostgreSQL and Redis.
2. Create and fill `.env`.
3. Install npm dependencies.
4. Generate the Prisma client.
5. Apply database migrations.
6. Build the Rust engine and set `ENGINE_BINARY_PATH`.
7. Bootstrap the initial professor account.
8. Start the API.
9. Start the worker.
10. Start the frontend.

8.2 Step-by-Step Commands

8.2.1 1. Start PostgreSQL and Redis

```
docker compose up -d postgres redis
```

8.2.2 2. Install dependencies and generate Prisma client

```
npm ci
npm run prisma:generate
```

8.2.3 3. Apply migrations

```
npm run db:migrate:dev
```

8.2.4 4. Build the engine

```
cargo build --manifest-path engine/Cargo.toml --release
```

8.2.5 5. Set engine and storage paths

At minimum, make sure these values are present in `.env`:

```
ENGINE_BINARY_PATH=./engine/target/release/engine-cli
OBJECT_STORAGE_ROOT=./var/object-storage
NEXT_PUBLIC_API_BASE_URL=http://localhost:3001
```

8.2.6 6. Bootstrap the first professor

```
ADMIN_BOOTSTRAP_EMAIL=admin@example.com
ADMIN_BOOTSTRAP_PASSWORD=ChangeMe123!
npm run db:bootstrap-admin
```

8.2.7 7. Start the API

Use a dedicated terminal:

```
npm run dev --workspace @similarity/api
```

8.2.8 8. Start the worker

Use a second terminal:

```
npm run dev --workspace @similarity/worker
```

8.2.9 9. Start the frontend

Use a third terminal:

```
npm run dev --workspace @similarity/web
```

8.3 Development Access Points

Once all three services are running:

- frontend: http://localhost:3000
- API health/live: http://localhost:3001/health/live
- API health/ready: http://localhost:3001/health/ready

Chapter 9

Running with Docker

9.1 What the Compose File Starts

The repository's `docker-compose.yml` defines:

- `postgres`
- `redis`
- `api`
- `worker`
- `web`
- `caddy`

9.2 Important Service Relationships

- **API** depends on PostgreSQL, Redis, and the shared object-storage volume.
- **Worker** depends on PostgreSQL, Redis, the shared object-storage volume, and the bundled engine binary inside its container.
- **Web** depends on the API base URL supplied at build time.
- **Caddy** fronts the web and API and exposes ports 80 and 443.

9.3 Persistent Volumes

The Compose file defines volumes for:

- PostgreSQL data,
- Redis data,
- shared object storage,

- Caddy data,
- and Caddy config.

The most important application-specific volume is the shared object-storage volume mounted at:

- `/app/var/object-storage` in the API container
- `/app/var/object-storage` in the worker container

9.4 Recommended Production-Style Startup Sequence

9.4.1 1. Prepare `.env`

Set production-appropriate values, especially:

```
NODE_ENV=production
DATABASE_URL=postgresql://postgres:postgres@postgres:5432/similarity_app?schema
    ↪ =public
REDIS_URL=redis://redis:6379
CORS_ALLOWED_ORIGINS=https://your-domain.example
NEXT_PUBLIC_API_BASE_URL=https://your-domain.example/api
AUTH_COOKIE_SECURE=true
PUBLIC_UPLOAD_STATUS_TOKEN_SECRET=<secure-random-secret>
SUBMISSION_IDENTITY_KEY_ID=<key-id>
SUBMISSION_IDENTITY_PUBLIC_KEY_PEM_BASE64=<base64-pem>
SUBMISSION_IDENTITY_PRIVATE_KEY_PEM_BASE64=<base64-pem>
ADMIN_BOOTSTRAP_EMAIL=<bootstrap-email>
ADMIN_BOOTSTRAP_PASSWORD=<bootstrap-password>
```

9.4.2 2. Start only the infrastructure first

```
docker compose up -d postgres redis
```

9.4.3 3. Apply migrations and bootstrap admin

From the host:

```
npm ci
npm run prisma:generate
npm run db:migrate:deploy
npm run db:bootstrap-admin
```

9.4.4 4. Build and start the full stack

```
docker compose up --build -d
```

9.5 Docker Build Details

The production worker image performs two important build stages:

1. it compiles the Rust engine with `cargobuild--release`,

2. then copies the binary into the runtime image at `/app/bin/engine-cli`.

This means the worker container is self-contained and does not need a separate engine service.

Chapter 10

System Integration Flow

10.1 Practical End-to-End Assembly

The most important runtime flow for installation validation is the comparison workflow. In operational terms, it works as follows:

1. A user interacts with the **frontend**.
2. The frontend sends HTTP requests to the **API**.
3. The API stores upload metadata in **PostgreSQL** and writes incoming archive bytes to staged shared **object storage**, then finalizes the stored archive before follow-up processing.
4. The API publishes a preparation or comparison job to **Redis/BullMQ**.
5. The **worker** consumes the job, reads the relevant files from object storage and database records, and prepares the engine request. During preparation, the finalized archive is read back from shared storage before extraction/preparation continues.
6. For comparison jobs, the worker launches the **Rust engine CLI**, sends JSON over stdin, and receives comparison results on stdout.
7. The worker writes comparison results back into **PostgreSQL**.
8. The frontend polls the API and renders the updated run status, suspicious pairs, and pair evidence.

10.2 Representative Upload-to-Result Flow

10.3 Why Startup Order Matters

The dependency chain during installation is strict:

- the API cannot become ready until PostgreSQL and Redis are reachable,

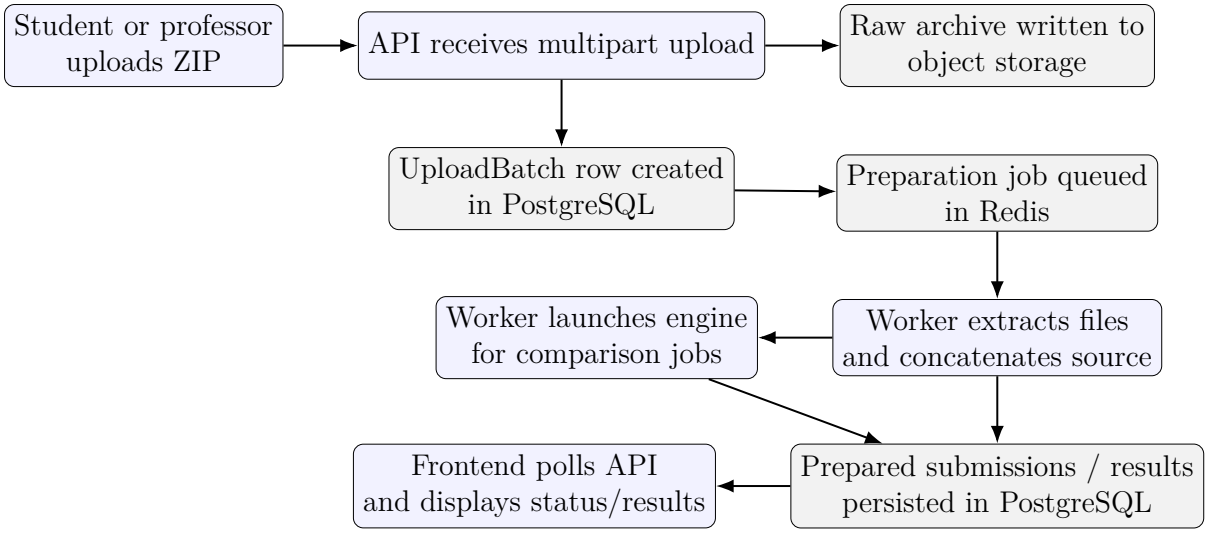

Figure 10.1: Practical integration flow from upload to review

- the worker cannot start successfully until PostgreSQL, Redis, object storage, and the engine binary are available,
- and the frontend is only useful once the API URL points to a working API instance.

Chapter 11

Verification and Smoke Testing

11.1 Infrastructure Health Checks

11.1.1 API liveness

Listing 11.1: Liveness check

```
curl -s http://localhost:3001/health/live
```

Expected response:

```
{"status":"ok","service":"api"}
```

11.1.2 API readiness

Listing 11.2: Readiness check

```
curl -s http://localhost:3001/health/ready
```

Expected response:

```
{
  "status": "ok",
  "checks": {
    "database": "ok",
    "redis": "ok"
  }
}
```

11.2 Application Smoke Test Sequence

11.2.1 1. Professor authentication

- Open http://localhost:3000/login
- Sign in with the bootstrapped professor account
- Confirm the dashboard loads without API errors

11.2.2 2. Assignment creation

- Create a new assignment from the professor dashboard
- Confirm that an assignment key is shown
- Confirm the assignment appears in the dashboard list

11.2.3 3. Upload preparation

- Open the student submission page at http://localhost:3000/
- Submit a ZIP file using the assignment key
- Confirm the upload status progresses from received/processing to ready

11.2.4 4. Comparison run

- Return to the professor assignment workspace
- Confirm at least two non-template submissions exist
- Run a comparison
- Confirm the run status is reflected correctly in the UI and progresses appropriately for the current system state

11.2.5 5. Pair review

- Open one suspicious pair
- Confirm that side-by-side evidence renders
- Confirm highlighted spans and navigation work

11.3 Optional Docker Verification

If running behind Caddy in a production-style deployment, use the public API path instead of the direct API port:

Listing 11.3: Example production-style health checks

```
curl -s https://your-domain.example/api/health/live
curl -s https://your-domain.example/api/health/ready
```

Chapter 12

Deployment Notes

12.1 Development vs Production

Topic	Development	Production-style deployment
Frontend API URL	usually `http://localhost:3001`	public URL such as `https://domain/api`
Cookie security	often `AUTH_COOKIE_SECURE=false`	must be `true` behind HTTPS
Identity keys	runtime defaults may exist	provide explicit real keys
Public status secret	development fallback exists	provide explicit secret
Engine path	local filesystem path	container path `/app/bin/engine-cli`
Object storage	local directory	persistent mounted volume
Reverse proxy	optional	Caddy or equivalent HTTPS front door

12.2 Security Changes Required Before Real Deployment

- Set `NODE_ENV=production`.
- Set `AUTH_COOKIE_SECURE=true`.
- Set `AUTH_COOKIE_DOMAIN` appropriately for the public domain.
- Set `CORS_ALLOWED_ORIGINS` to the exact frontend origins.
- Use real values for the public-upload secret and RSA identity key pair.
- Use HTTPS and make sure `NEXT_PUBLIC_API_BASE_URL` points at the public API route exposed through the reverse proxy.

12.3 Common Misconfigurations

- **API URL mismatch**: frontend built with the wrong `NEXT_PUBLIC_API_BASE_URL`.

- **Missing shared storage**: API and worker point at different `OBJECT_STORAGE_ROOT` locations.
- **Engine path missing**: worker cannot locate `ENGINE_BINARY_PATH`.
- **Database host mismatch**: Compose deployments should use the container hostname in `DATABASE_URL`, not `localhost`.
- **Cookie misconfiguration**: secure cookies enabled without HTTPS, or wrong cookie domain, causing login/session failures.
- **Migrations skipped**: services start, but API operations fail because the database schema was never deployed.

12.4 Final Assembly Checklist

Before handing the system to users, confirm all of the following:

1. PostgreSQL is reachable and migrated.
2. Redis is reachable.
3. Prisma client has been generated.
4. The Rust engine has been built.
5. `ENGINE_BINARY_PATH` points at the actual binary.
6. API and worker share the same object-storage root.
7. The initial professor account exists.
8. Health endpoints return `ok`.
9. Student upload and professor comparison flows work end-to-end.